校园资料AI检索助手 技术文档

1. 项目概述

1.1 项目背景

在校园日常学习与办公中，课程课件、培养方案、考试通知、实验报告等资料散落在班级群、个人U盘、网盘等各处，存在以下典型痛点：

痛点	具体表现	传统解决方式的局限
资料分散	同一门课的课件可能分布在多个群文件、多个同学的U盘中	每次找资料需要翻多个渠道，效率极低
查找困难	只能靠"文件名记忆"去找，记不清文件名就找不到	没有内容级检索能力，无法按语义搜索
版本混乱	同一份培养方案被反复修改，群里同时存在多个版本	无法区分新旧版本，容易用错过期文件
信息无法问答	培养方案里的学分要求、考试通知里的细则需要人肉翻阅	文档只能"看"，不能"问"

本项目正是针对上述痛点，构建一套**本地化部署的校园资料智能检索系统**：把校园文档统一托管、自动分类、语义检索，并基于RAG（检索增强生成）架构实现"带来源标注的智能问答"，全程使用本地开源模型，无需调用外部付费API，保障学生课件、培养方案等敏感校园数据不出校。

1.2 核心功能列表

- 多格式文档上传：支持PDF、Word、TXT格式，自动提取文本内容并入库
- AI自动文档分类：自动按校园场景归类，支持教师人工修正
- 带溯源标注的智能问答：回答自动标注参考来源文档，防幻觉
- SSE流式输出：回答以打字机效果逐字呈现，首字2-3秒可见，避免长时间空白等待
- 三角色账号体系：学生（自行注册）/ 老师（管理员创建）/ 高级管理员（固定账号），密码加盐哈希加密存储
- 问答历史留痕：每次提问自动保存，支持搜索、备注、修改权限
- 三档内容权限：私有（仅自己可见）/ 仅链接可见（免登录分享）/ 公开（展示个人主页）
- 分享链接与个人主页：生成分享链接，公开问答聚合展示在个人主页
- 文档版本管理：同名文档自动保留历史版本，优先展示最新版，支持一键回滚
- 教师文档管理：上传/删除/改分类/版本回滚归教师角色，按登录Token鉴权
- 管理员后台：用户管理（新增/删除老师账号，删除时级联清理其文档与问答）、公开内容总管理
- 教师评语：教师可对自己上传文档相关的公开问答写评语，学生历史/主页/分享页全端可见
- 分类折叠展示：文档库按分类折叠，界面简洁清晰
- 原文件查看：详情支持打开/下载原始文件，保留原始格式
- 前端工程化：Vue3 + Pinia + VueRouter + Element Plus 多页面应用，路由级鉴权

1.3 技术选型与依据

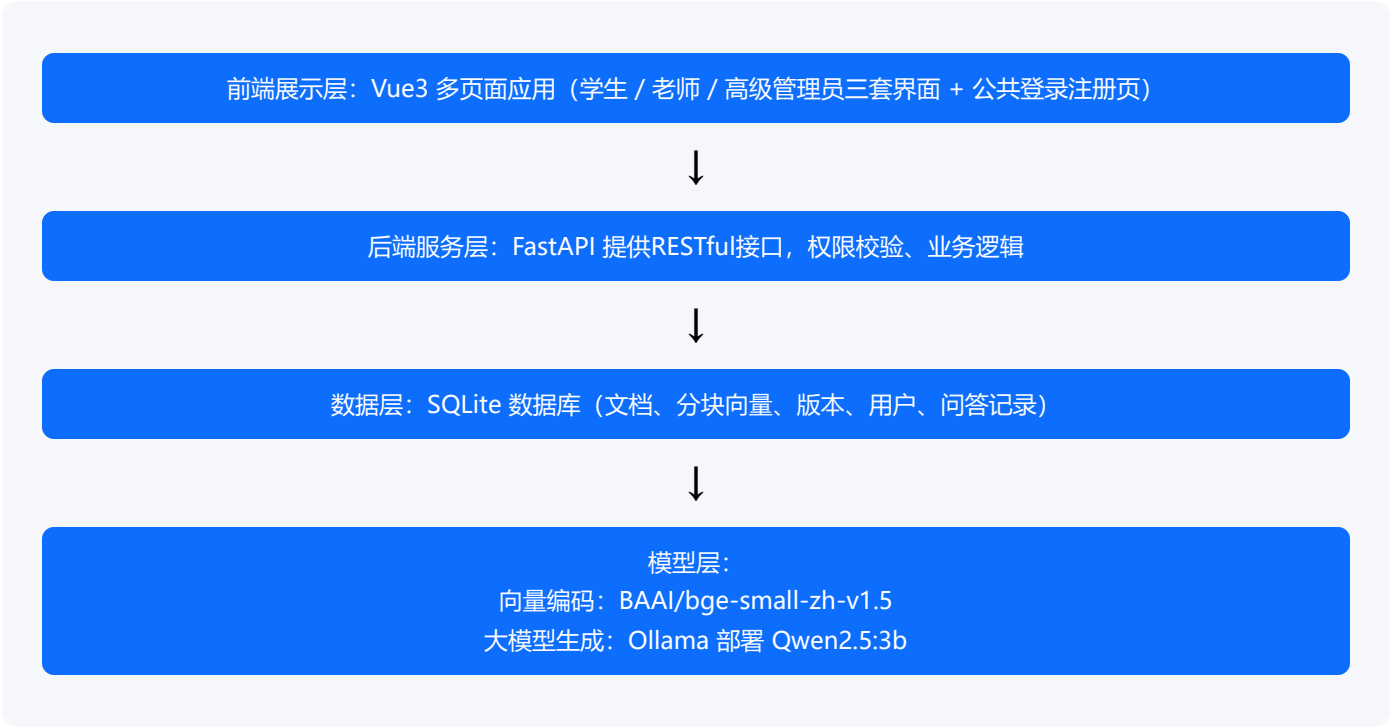
层级	技术组件	选型依据

后端框架	FastAPI + Uvicorn	轻量、性能高、自带接口文档（/docs），适合本地部署
数据库	SQLite（SQLAlchemy ORM）	零配置、单文件、免部署，满足轻量场景
向量编码	BAAI/bge-small-zh-v1.5	中文语义效果好、模型小（约90MB）、CPU可实时推理
大模型	Ollama 部署 Qwen2.5:3B	本地运行、数据不出校、3B参数在CPU上可运行
前端框架	Vue3 + Vite + Pinia + VueRouter	组件化多页面应用，状态管理统一，路由级权限控制
UI组件库	Element Plus	成熟Vue3组件库，表格/弹窗/表单开箱即用
文本解析	PyMuPDF / python-docx	分别解析PDF与Word文档，纯本地处理

2. 系统架构设计

2.1 总体架构

系统采用轻量前后端分离架构，无重型依赖，部署简单：



2.2 核心数据流程

入库链路：

```
用户上传文档 → 按类型提取文本(PDF/Word/TXT) → 按语义分块(每块约300字)
→ 向量编码(bge-small-zh) → 写入chunks表 → 调用大模型自动分类 → 写入documents表
```

问答链路：

用户提问 → 问题向量化 → 余弦相似度检索Top3文本块 → 相似度阈值判断(低分走正常对话)
→ 拼接上下文Prompt → 本地大模型流式生成(SSE逐字返回) → 前端打字机渲染 → 标注参考来源
→ 结束后自动存入问答历史

版本管理链路：

上传同名文档 → 检测同名最新版 → 弹出覆盖确认 → 旧版标记is_latest=0
→ 新文档version+1、parent_doc_id指向旧版 → 管理员可查看历史版本链并回滚

3. 数据库表结构设计

3.1 文档主表（documents）

字段名	类型	说明
id	整数(主键)	文档主键，自增
filename	字符串	用户上传的原始中文文件名
filetype	字符串	文件类型：pdf/word/txt
file_path	字符串	服务器存储的安全文件名路径（UUID命名防冲突）
content	长文本	提取后的文档全文内容
category	字符串	AI自动分类标签（培养方案/考试通知/课件资料/行政通知/社团文件/作业）
category_confirmed	整数	是否被管理员人工修正过（0=AI分类，1=人工修正）
version	整数	文档版本号，从v1开始递增
is_latest	整数	是否为当前最新版（1=最新，0=历史版），学生端仅展示最新版
parent_doc_id	整数	上一版本文档id，用于追溯历史版本链
uploaded_at	时间	文档上传时间

3.2 向量分块表（chunks）

字段名	类型	说明
id	整数(主键)	分块主键，自增
doc_id	整数	所属文档id，关联documents表
chunk_text	长文本	分块后的文本片段

embedding	文本(JSON)	文本片段对应的向量编码，用于相似度检索

3.3 关键设计说明

- **版本链设计**：通过 parent_doc_id 形成单向链表，配合 is_latest 标记实现"同名文档只展示最新版、可追溯历史版本链"的需求。为避免版本链循环引用（如parent指向自身），后端在回滚与查询时均增加了防循环遍历保护。
- **向量存储**：embedding字段以JSON字符串形式存储768维浮点向量，在轻量场景下避免了引入独立的向量数据库，降低部署复杂度。
- **索引设计**：对 is_latest、filename 字段建立查询索引，保证文档列表与同名检测的查询性能。

4. 核心模块实现

4.1 RAG检索与防幻觉模块（rag.py）

本模块负责文档分块、向量编码入库与相似度检索，是"能问答"的核心。

核心实现：

```
def search_top3_chunks(db_path: str, question: str) -> list:
    """问题向量化后与库中分块做余弦相似度匹配，返回Top3"""
    model = get_encoder()                       # 复用全局向量模型，避免重复加载
    q_vec = model.encode(question)              # 问题编码
    scored = []
    for chunk in all_chunks:                    # 遍历库中分块
        c_vec = json.loads(chunk["embedding"])
        sim = cosine_similarity(q_vec, c_vec)  # 余弦相似度
        scored.append((sim, chunk["text"], chunk["doc_id"]))
    scored.sort(reverse=True, key=lambda x: x[0])
    return [{"score": float(s), "text": t, "doc_id": d} for s, t, d in scored[:3]]
```

- 文本分块：按每300字左右拆分，保留语义完整性
- 相似度检索：余弦相似度匹配，返回最相关的3个文本块作为大模型上下文
- 防幻觉机制：Prompt强制约束大模型只能基于检索到的片段回答，无相关内容时直接返回"找不到信息"，禁止编造内容
- 闲聊识别：对检索相似度低于阈值的提问（如"你好"），自动切换为正常对话模式，避免无关文档被强行引用

4.2 本地大模型调用模块（llm.py）

通过OpenAI兼容协议调用Ollama本地部署的Qwen2.5:3B模型，实现回答生成、文档自动分类与**SSE流式输出。**

```
client = OpenAI(
    base_url="http://localhost:11434/v1",   # Ollama本地服务
    api_key="ollama",
    timeout=120.0,     # 本地CPU推理较慢，放宽超时
    max_retries=0      # 关闭自动重试，避免等待时间翻倍
)

def chat_stream(prompt: str):
    """流式生成：SSE逐块返回内容，首字2-3秒即可见"""
    response = client.chat.completions.create(
        model="qwen2.5:3b",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.3,
        stream=True,               # 开启流式
        max_tokens=600,
        extra_body={"keep_alive": -1}   # 模型常驻内存，避免每次重新加载
    )
    for chunk in response:
        if chunk.choices[0].delta.content:
            yield chunk.choices[0].delta.content
```

- 模型名与Ollama注册模型严格一致（qwen2.5:3b），通过 /api/tags 校验模型存在
- **流式提速**：stream=True 逐token返回，配合前端打字机渲染，闲聊首字约2.5秒、全程约4秒（原整段等待20秒+）
- **模型常驻**：keep_alive=-1 让模型常驻内存，避免每次提问重新加载；检索片段裁剪到300字/段，降低CPU prefill耗时
- 异常兜底：大模型无响应时直接返回检索到的文档片段，页面不卡死

4.3 AI自动分类模块

文档上传后自动取前500字内容，调用本地大模型将文档分到6类校园场景：培养方案、考试通知、课件资料、行政通知、社团文件、作业。同时提供管理员人工修正接口，修正后的标注数据可用于后续模型持续微调。

```
def classify_document(doc_content: str) -> str:
    prompt = f"""你是校园文档分类助手，把下面这份文档分到以下6类中的一类：
    可选分类：培养方案、考试通知、课件资料、行政通知、社团文件、作业
    作业指学生提交的实验报告、课程作业、用例规约等；课件资料指老师上课用的PPT、讲义、教案。
    只返回分类名称。
    文档内容：{preview}
    分类结果："""
    # 优先精确匹配 → 同义词兜底（"实验/作业/用例"→作业，"课件/讲义/教案"→课件资料）→ 未分类
```

- 分类提示词中明确"作业"与"课件资料"的边界定义，提升分类准确率
- 增加同义词兜底匹配：模型回答不标准时也能正确归类，避免"未分类"堆积

4.4 权限与版本管理模块

- **三角色权限**：登录Token鉴权，学生/老师/高级管理员三角色隔离；文档管理归教师角色，私有问答任何角色不可见（详见第14章）
- **版本管理**：同名文档重复上传时，先查同名最新版（避免查到刚插入的自身），弹出覆盖确认后，旧版本标记is_latest=0，版本号递增，学生端仅展示最新版，管理员可查看完整历史版本链并一键回滚

```
# 上传时的版本升级核心逻辑（先查旧版，再保存新文档）
old = db.query(Document).filter(
    Document.filename == file.filename,
    Document.is_latest == 1
).first()
doc = save_document(db, file.filename, filetype, file_path, content)
if old:
    old.is_latest = 0            # 旧版转为历史版
    doc.version = old.version + 1
    doc.parent_doc_id = old.id   # 建立版本链
```

4.5 前端交互模块（Vue3工程）

Vue3多页面应用（frontend/），按角色组织界面：

- **公共页**：登录、注册（仅学生）、分享落地页、个人主页
- **学生界面**：AI问答（带参考来源标注）+ 文档库（分类折叠、搜索、排序、详情）+ 问答历史（备注/权限/分享）+ 个人主页 + 个人设置
- **老师界面**：AI问答 + 文档管理（上传、删除、改分类、历史版本、回滚）+ 公开内容管理
- **管理员界面**：后台首页 + 用户管理（新增/删除老师账号）+ 公开内容总管理
- **上传体验**：同名文档上传时弹窗确认"是否覆盖升级"，防止误覆盖
- **原文件查看**：详情弹窗支持"打开原文件"，PDF直接预览、Word下载保留原格式

5. 后端接口设计

系统共提供20+个RESTful接口，均通过FastAPI实现，自带Swagger文档（/docs）。

POST **/upload/ — 上传文档（教师，Bearer Token鉴权）**

参数：file（文件）、confirm（同名覆盖确认标志）；鉴权：Authorization: Bearer <token>（教师角色）

流程：校验教师身份 → 校验格式(PDF/Word/TXT) → 检测同名 → 提取文本 → 保存入库（记录teacher_id归属）→ 分块向量化 → AI自动分类

返回：{msg, filename, version, category}

GET **/documents/ — 文档列表（公开）**

说明：返回所有最新版文档（is_latest=1），按上传时间倒序，学生端数据源

返回：[{id, filename, category, version}]

GET **/documents/search/ — 按文件名搜索（公开）**

参数：keyword（模糊匹配文件名）

返回：匹配的文档列表

GET **/documents/{id} — 文档详情（公开）**

说明：返回文档元数据与提取的全文内容，供详情弹窗展示

GET **/file/{id} — 原文件查看/下载（公开）**

说明：按类型返回原始文件（PDF直接预览、TXT直接显示、Word下载），保留原始格式

DELETE **/documents/{id} — 删除文档（教师，Token鉴权）**

鉴权：Bearer Token（教师角色）
说明：删除数据库记录并清理服务器上的原始文件

PUT **/admin/documents/{id}/category — 人工修正分类（教师，Token鉴权）**

参数：new_category
鉴权：Bearer Token（教师角色）
说明：教师手动调整AI分类结果，标记category_confirmed=1

GET **/admin/documents/{id}/versions — 历史版本链（教师，Token鉴权）**

说明：通过parent_doc_id递归遍历同一文件名的所有历史版本，支持一键回滚

POST **/ask/ — RAG智能问答（登录用户，SSE流式输出，自动留痕）**

参数：q（问题文本）；鉴权：Bearer Token（登录用户）

流程：问题向量化 → Top3检索 → 相似度阈值判断（低分走正常对话）→ 拼接Prompt → 本地大模型**流式生成**（SSE协议：data: {"c":"..."} 逐块 → data: {"sources":[...]} → data: [DONE]）→ 前端打字机渲染 → 结束后自动保存问答记录（默认私有）

POST **/register/ — 学生注册**

参数：username、password（≥4位）；角色固定为 student，用户名唯一校验，密码加盐SHA256存储

POST **/login/ — 登录**

参数：username、password；返回 token + role，token 写入 users 表，禁用账号拒绝登录

GET **/me/ — 当前登录用户信息**

鉴权：Bearer Token；返回 {user_id, username, role}

GET **/qa/history/ — 我的问答历史**

参数：keyword（可选，搜索问题/答案）；返回本人全部问答记录

PUT **/qa/{id}/note — 写备注**

说明：给本人问答记录添加备注，备注可见性跟随问答权限

PUT **/qa/{id}/permission — 修改权限**

参数：permission（private / link / public）；私有问答不允许生成分享链接

POST **/qa/{id}/share — 生成分享链接**

说明：返回分享token，供落地页 /qa/share/{token} 使用（免登录）

GET **/users/{id}/profile 与 /users/{id}/qa_public — 个人主页**

说明：免登录查看用户公开信息与其公开问答列表

POST **/change_password/ — 修改密码**

参数：old_password、new_password；校验原密码后重新加盐哈希存储

GET **/teacher/qa/public/ — 教师查看学生公开问答**

鉴权：Bearer Token（教师角色）；仅返回学生 role 的公开问答

PUT **/teacher/qa/{qa_id}/note — 教师写评语**

鉴权：Bearer Token（教师角色）；参数：note（评语内容，留空清除）

说明：仅可对自己上传文档关联的公开问答写评语（归属校验同删除接口）；评语与学生自己的备注（note）相互独立，展示在学生历史/个人主页/分享页

DELETE **/qa/{id} — 教师删除关联公开问答**

鉴权：Bearer Token（教师角色）；校验问答为公开权限且来源文档归属于该教师（teacher_id匹配）

GET **/admin/users/ — 管理员查看全部用户**

鉴权：Bearer Token（admin角色）；返回全部用户及账号状态

POST **/admin/teachers/ — 管理员新增老师账号**

鉴权：Bearer Token（admin角色）；创建 role=teacher 账号

DELETE **/admin/users/{id} — 管理员删除账号**

鉴权：Bearer Token（admin角色）；说明：直接删除账号并**级联清理**该用户上传的文档与该用户全部问答记录；不能删除自己、不能删除admin角色

GET **/admin/qa/public/ 与** **DELETE** **/admin/qa/{id} — 管理员公开内容总管理**

鉴权：Bearer Token（admin角色）；查看/删除全部用户公开问答

6. 开发过程Bug记录与修复（AI协同核心证据）

本项目全程使用大模型辅助开发，以下记录真实开发过程中遇到的典型Bug与AI协同定位修复的过程，是"人机协同开发"能力的最直接证据。

Bug1：新开Git Bash窗口找不到uvicorn命令
现象：每次新开终端执行uvicorn都报 command not found
排查：uvicorn安装在项目虚拟环境(venv)中，新开终端未激活虚拟环境，Python包不在PATH中
修复：编写start.sh一键启动脚本，自动激活虚拟环境、配置HF镜像、启动服务，后续无需手动操作
经验：任何需要"每次手动重复"的环境操作都应脚本化，这也是项目可复现性的基础

Bug2：下载向量模型连接HuggingFace官网超时
现象：加载模型时报WinError10060连接失败，模型无法下载
排查：HuggingFace官网在国内网络环境下连接不稳定
修复：配置国内HF镜像端点 export HF_ENDPOINT=https://hf-mirror.com，模型下载速度从超时变为秒下
经验：国内环境使用开源模型必须考虑镜像加速，该配置已写入README复现指南

Bug3：新上传的文档学生列表看不到
现象：上传接口返回成功，但是前端文档列表不显示新文档
排查：学生端列表查询条件为 is_latest==1，而新文档保存时该字段未显式赋值，SQLAlchemy默认值为NULL，被过滤掉
修复：保存文档时显式设置 doc.is_latest = 1，上传后立即生效
经验：ORM字段默认值不可靠时，必须显式赋值，尤其是与查询条件强耦合的字段

Bug4：调用大模型时页面一直转圈无响应
现象：点击提问后前端长时间加载，最终无响应
排查：本地CPU推理较慢，且OpenAI SDK默认自动重试2次，30秒超时+2次重试导致总等待时间超过90秒
修复：超时放宽到120秒、关闭SDK自动重试(max_retries=0)、增加异常兜底直接返回检索片段
经验：本地推理场景必须按实际算力调整超时策略，不能沿用云端API默认配置

Bug5：前端跨域请求被浏览器拦截
现象：fetch调用后端接口报CORS错误
排查：前端以http://127.0.0.1:8000访问，后端未配置跨域许可
修复：在FastAPI中添加CORSMiddleware中间件，允许跨域访问
经验：前后端分离架构必须显式配置CORS，属于常见且必要的基础配置

Bug6：Ollama模型识别失败（404）
现象：问答返回"本地大模型暂时无响应"，日志显示chat/completions返回404 Not Found
排查：通过 /api/tags 查询发现模型列表为空。进一步排查发现：环境变量OLLAMA_MODELS指向D盘目录，而D盘只有下载中断的partial残片且缺少config/template/license/params等4个关联文件，模型库不完整
修复：从C盘旧模型目录复制完整模型（5个blob文件）到D盘目录，重启Ollama后 /api/tags 正常识别qwen2.5:3b
经验：Ollama模型库包含manifest注册文件与多个blob文件，任何一个缺失都会导致模型不被识别；模型迁移必须整体复制，不能只拷权重

Bug7：问答20秒+等待，演示录视频等不起
现象：每次提问要等20多秒才出结果，演示录屏8分钟根本录不完
排查：接口是"整段生成完再返回"；本地CPU推理慢，且Ollama模型每次调用后可能被卸载，二次提问要重新加载权重
修复：①/ask/改SSE流式输出，首字2-3秒就开始显示（打字机效果）；②keep_alive=-1让模型常驻内存；③检索片段裁剪到300字/段降低prefill耗时；④max_tokens限制单次回答长度。实测闲聊首字2.5秒/全程4秒，文档问答全程33秒降到7秒
经验：本地CPU推理场景，"流式输出+模型常驻+上下文裁剪"三件套是演示体验的关键

Bug8：权限迁移后教师管不了历史文档
现象：文档管理从admin迁移到教师后，教师删除/写评语全部返回403"只能管理自己上传的文档"
排查：documents表新增的teacher_id字段在迁移补列后全部为NULL，旧文档没有归属，归属校验全部失败
修复：将已有教学文档的teacher_id统一回填到teacher账号（UPDATE documents SET teacher_id=2 WHERE teacher_id IS NULL）
经验：表结构迁移补列时，存量数据的归属/外键字段必须一并回填，否则新校验逻辑会对旧数据"一票否决"

Bug9：前端美化后登录注册全部404
现象：界面重构+Vite代理改造后，登录注册接口全部404，页面无法使用
排查：vite代理规则使用带$锚定的正则（^/login/$），浏览器带query串的POST请求（/login/?username=..）不命中正则，被SPA回退消化
修复：代理规则全部改为字符串前缀写法（'/login/'、'/register/'、'/admin/users/'等），POST带query也能正确转发到后端
经验：Vite代理正则的$锚定会漏掉带query的POST请求，用字符串前缀匹配更稳妥；前端路由不带尾斜杠、后端API带尾斜杠可天然避免冲突

7. AI协同开发策略

本项目全程使用大模型辅助开发，核心Prompt设计思路：

1. **小步迭代**：一次只实现一个功能模块，跑通验证后再开发下一个，避免一次性生成大量代码出错
2. **明确边界**：每次需求都写清输入输出，避免AI生成多余代码
3. **根因定位**：遇到报错直接把终端堆栈信息反馈给AI，引导AI定位根因修复，不盲目改代码
4. **可复现约束**：要求AI写的代码无特殊环境依赖，所有依赖写入requirements.txt，环境变量单独配置
5. **证据留痕**：所有开发对话均保留完整记录（见Prompt证据链文档），每轮开发有明确的功能目标与验证方式
6. **意图控制**：给AI的Prompt全部包含"角色定义+任务边界+输出格式+禁止事项"四要素。例如文档分类Prompt明确"只返回分类名称"、问答Prompt明确"只能基于检索片段回答，无相关内容返回找不到信息，禁止编造"——从源头约束AI不瞎编

7. **量化兜底**：对AI输出做二次校验（相似度阈值0.45区分问答/闲聊、同义词兜底分类、来源文档标注），AI输出不可靠时用程序逻辑兜底，保证最终产物符合要求

Prompt证据链说明：项目在backend/prompt-evidence目录下保留了3份核心Prompt开发证据：

证据文件	覆盖功能模块	证据要点
prompt-evidence-1-backend-rag.md	后端基础与RAG检索	文档解析、向量检索、自动分类、版本管理，6个Bug修复
prompt-evidence-2-local-llm.md	本地大模型与流式问答	Ollama部署、闲聊阈值、SSE流式提速，6个Bug修复
prompt-evidence-3-users-frontend.md	用户权限与前端工程化	三角色、留痕分享、权限迁移、评语删除、Vue3美化，11个Bug修复

以下为AI协同开发过程中的真实对话记录截图：

7.1 项目立项与第一条正式Prompt



7.2 Embedding模型选型（本地开源方案论证）

现在，我帮你写一条**强约束 Prompt**，你可以拿去给 AI 编程工具用，同时这条 Prompt 本身就是你的参赛证据。

你先确认几个点：

1. Embedding 模型你想用**本地开源模型**（离线可用，但要下载模型文件），还是**调用 API**（比如通义千问 / OpenAI 的 Embedding 接口，不用下载模型，但需要 API Key）？
2. 大模型问答部分，你打算用什么？还是先把检索部分做出来，问答接口后面再接？

确认完这两个，我就把完整的强约束 Prompt 写给你。

哪个好一点

对比两种方案，结合你的参赛、复现简单、少踩坑需求来选

方案 A：本地 Embedding（bge-small-zh，推荐给你）

✅优点

1. **不需要 API 密钥、不用花钱、不限调用次数**，复现的时候不用配置 key，评委拉代码直接就能跑，**对比赛可复现性非常友好**
2. 离线运行，不依赖外网，不会遇到接口超时、额度耗尽的问题
3. 中文适配好，非常适合文档知识库检索

输入问题或任务，/ 选择技能

工作任务 按需确认 技能 连接器 自动 高

7.3 后端接口开发完成核对



打算的功能：文档上传、文件检索、基于文档内容问答（涉及 ai）、后台自动文件分类
感觉有些简单，可以增加一些点，比如：
1. 文档分类：不要做传统的只用关键词分类，可以根据文档内容提取（可以引入文本分类模型，小的智能体啥的）。也可以按校园类目自动归类：比如培养方案、考试通知、课件、社团文件、行政通知；可以支持自定义分类标签、人工修正分类结果，如果可以的话可以支持模型的持续微调和改进
2.ai 回答问题的时候，建议不要简单的回答问题，一般 ai 思索的时候会有依据，可以自动标注下相关依据，并告诉用户回答的是哪一页或者哪一部分内容
3. 文档版本管理，建议保留几个版本，优先展示最新的版本。万一传错版本了，方便回滚
4. 要做一些权限管理，比如普通学生只能查询问答，然后知识库文档库可以管理员上传
指导老师给我的建议

梳理课程项目升级方案 >

老师这几个建议非常准，完全不用你搞什么大模型微调那种重活，咱们顺着你现有已经跑通的 RAG 基础，**把这几个点包装成轻量可落地的功能**，两周内就能做完，直接从 "跑通 demo" 升级成 "完整校园知识库产品"，答辩一下子就有亮点了。

我帮你把老师的建议拆成「能真做、能出效果、不加班」的优先级方案

输入问题或任务，/ 选择技能
+ 工作任务 按需确认 技能 连接器 自动 高

全国高校Vibe Coding开发挑战赛介绍
LAPTOP-LQMGSNC4
1

基于我现有的 FastAPI 项目。现有项目结构：main.py、database.py，使用 SQLite。
任务：新建独立文件 rag.py，实现 RAG 分块、embedding 生成、余弦相似度检索函数。禁止修改 main.py、database.py 已有代码。
技术约束：
1. 使用 bge-small-zh-v1.5 作为 embedding 模型
2. 文本分块：chunk_size=500，chunk_overlap=100
3. 向量结果转为 json 字符串，存入 sqlite 的 chunks 表
4. 新增数据库表 chunks，字段：id, doc_id（关联文档表）, chunk_text, embedding
5. 实现余弦相似度计算函数，检索返回 top3 最相关 chunk
6. 增加异常捕获，文档为空、模型加载失败返回标准 JSON 错误
7. 只输出代码，附带简短注释，不要重构原有模块。

python 运行

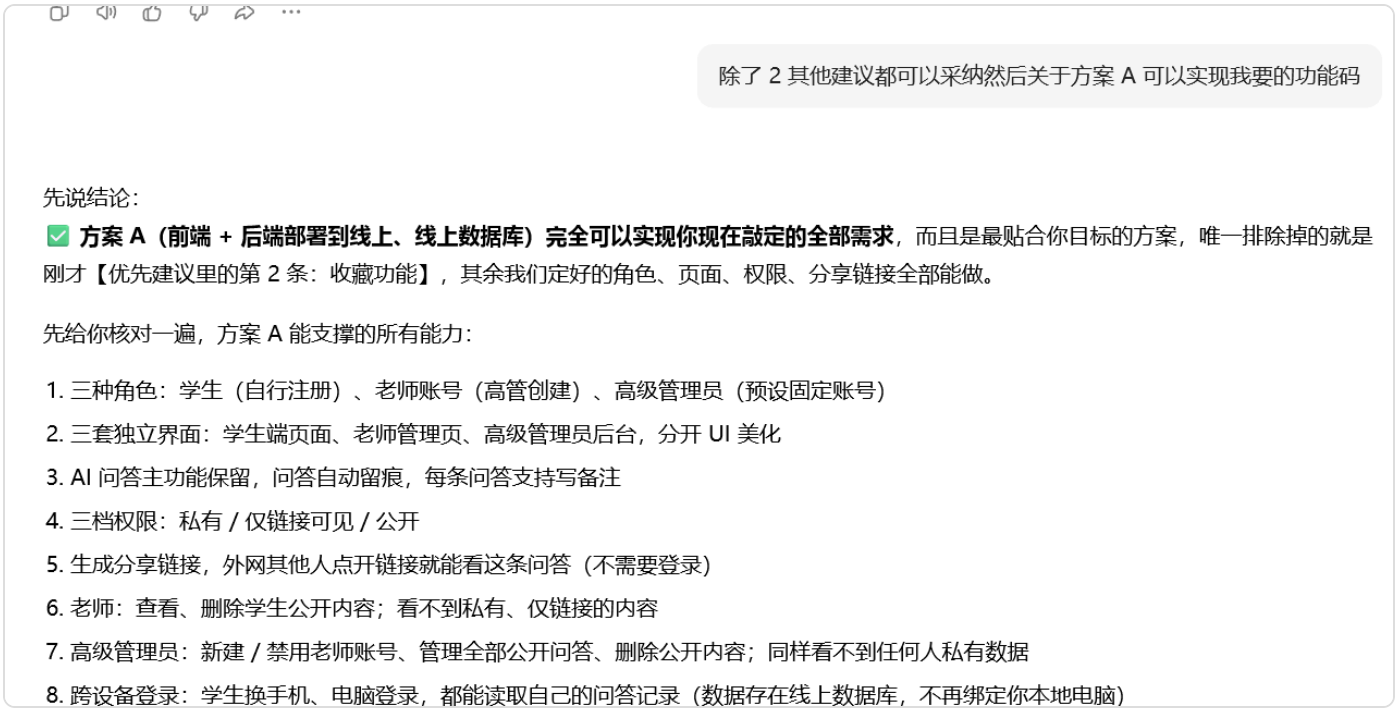
```python
# rag.py
import json
import numpy as np
from sentence_transformers import SentenceTransformer
import sqlite3
```

输入问题或任务，/ 选择技能
+ 工作任务 按需确认 技能 连接器 自动 高

7.4 用户需求梳理与部署方案确认



向AI一次性提出注册/独立空间/问答留痕/备注/权限控制/分享链接/超级管理员等完整需求，AI自动梳理成结构化需求清单，逐条确认实现范围。



确认线上部署方案A可支撑全部需求，敲定三角色体系、三套独立界面、三档权限、分享链接与跨设备登录的能力边界——这是后续所有功能开发的需求基线。

8. 系统界面展示

8.1 新版多角色界面（Vue3工程）

登录页（账号体系入口，学生/老师/高级管理员三角色）



学生AI问答界面（智能问答 + 文档库分类折叠 + 个人导航）



问答历史页（自动留痕、备注、三档权限、生成分享链接）



教师界面（AI问答 + 文档库 + 教师上传文档）



管理员后台-用户管理（新增老师账号、删除账号、用户列表）



管理员后台-公开内容总管理（查看/删除全部用户公开问答）



Git提交历史（细粒度commit记录，全程AI协同开发留痕）



9. 测试与验证

9.1 功能测试用例

编号	测试项	操作步骤	预期结果	结果
T01	上传PDF文档	管理员上传PDF文件	上传成功、自动分类、显示版本v1	✅ 通过
T02	上传Word文档	管理员上传.doc/.docx文件	文本提取成功、自动分类	✅ 通过
T03	上传TXT文档	管理员上传txt文件	文本提取成功、自动分类	✅ 通过
T04	同名文档版本升级	再次上传同名文件并确认	版本号+1、旧版转为历史版	✅ 通过
T05	同名上传取消	上传同名文件点"取消"	不上传、旧版本保持不变	✅ 通过
T06	版本回滚	管理员查看历史版本并回滚	回滚到指定版本、旧最新版转历史	✅ 通过
T07	智能问答	学生提问文档相关内容	返回答案并标注参考来源	✅ 通过
T08	闲聊识别	提问"你好"	正常问候，不引用无关文档	✅ 通过
T09	权限校验	学生访问上传接口	返回403权限不足	✅ 通过

T10	分类修正	管理员修改文档分类	分类更新并标记人工修正	✅ 通过
T11	文档搜索	按关键词搜索文件名	返回模糊匹配结果	✅ 通过
T12	原文件查看	详情弹窗点击"打开原文件"	PDF预览/Word下载，格式保留	✅ 通过
T13	学生注册	注册页提交新用户名+密码	注册成功自动登录，密码加密存储	✅ 通过
T14	三角色登录	分别用学生/老师/管理员账号登录	跳转各自角色首页，界面菜单不同	✅ 通过
T15	问答自动留痕	学生提问后查看问答历史	记录自动保存，默认私有，显示来源	✅ 通过
T16	备注与权限修改	历史页写备注、改权限为公开	备注保存，问答出现在个人主页	✅ 通过
T17	分享链接	设为仅链接可见并生成分享链接	他人免登录打开落地页查看该问答	✅ 通过
T18	个人主页	访问学生公开主页 /u/{id}	展示该学生全部公开问答，免登录	✅ 通过
T19	教师文档管理	教师登录上传/删除/改分类文档	Token鉴权放行，admin不再有文档管理	✅ 通过
T20	教师删除关联公开问答	教师删除引用其文档的公开问答	校验归属通过删除；无关问答拒绝403	✅ 通过
T21	管理员用户管理	管理员新增/删除老师账号	新增成功；删除后账号消失且级联清理其数据	✅ 通过
T22	管理员删任意公开问答	管理员删除任意学生公开问答	删除成功；私有问答拒绝删除	✅ 通过
T23	流式问答输出	学生提问后观察回答呈现	打字机逐字输出，首字2-3秒可见，回答完整落库留痕	✅ 通过
T24	教师写评语	教师对自己文档关联的公开问答写评语	评语保存；学生历史/主页/分享页可见；无关问答403	✅ 通过
T25	用户删除级联清理	管理员删除老师账号后检查其文档与问答	账号删除后其上传文档与问答记录一并清理	✅ 通过

9.2 异常场景验证

编号	异常场景	处理策略	结果	
E01	大模型无响应/超时	120秒超时+兜底返回检索片段，页面不卡死	✅ 通过	
E02	Ollama模型丢失	接口返回404，兜底提示，不崩溃	✅ 通过	
E03	上传不支持格式	返回"只支持PDF/Word/TXT"提示	✅ 通过	
E04	版本链死循环	后端遍历加seen集合防循环	✅ 通过	
E05	向量模型加载失败	启动时输出明确错误日志	✅ 通过	
E06	角色越权访问	学生直接访问/teacher、/admin路由	前端守卫拦截跳回学生首页	✅ 通过
E07	未登录访问受保护接口	不带Token调用/ask/、/qa/history/	返回401未登录	✅ 通过
E08	教师写评语无归属	教师对非自己文档关联的公开问答写评语	返回403"只能管理自己上传文档关联的问答"	✅ 通过
E09	修改权限非法值	提交权限为其他字符串	校验拒绝，保持原权限	✅ 通过
E10	管理员删除保护账号	管理员删除自己或admin角色	返回400"不能删除自己的账号"	✅ 通过
E11	SSE流式连接中断	生成过程中前端断开	后端结束流并保存已生成内容，前端恢复后可见	✅ 通过

10. 部署与运行说明

10.1 环境要求

项目	要求
操作系统	Windows 10/11（本项目开发环境）
Python	3.10+（含venv虚拟环境）
大模型运行时	Ollama（本地部署Qwen2.5:3B）
Node.js	18+（前端Vue3工程构建/开发）

磁盘空间	模型约1.9GB + 项目约300MB
网络	首次安装需联网下载模型，运行时完全离线

10.2 启动与访问（前后端分离）

① 启动后端（FastAPI）：

```
cd backend
venv\Scripts\activate                # Windows；Git Bash: source venv/Scripts/activate
export HF_ENDPOINT=https://hf-mirror.com
uvicorn main:app --host 0.0.0.0 --reload --port 8000
```

② 启动前端开发模式（Vue3，另一终端）：

```
cd frontend
npm install      # 首次运行安装依赖
npm run dev      # 访问 http://localhost:5173（已代理到8000后端）
```

③ 预设账号：高级管理员 admin / admin123；老师 teacher / teacher123；学生自行注册。

④ 局域网访问：后端绑定 0.0.0.0 后，同一WiFi下其他设备通过 http://本机IP:8000 或 http://本机IP:5173 访问。

⑤ 生产部署：前端 npm run build 生成 dist，由后端或静态服务器托管。

10.3 Ollama模型配置要点

- 模型安装：ollama pull qwen2.5:3b（约1.9GB，支持国内镜像加速）
- 模型目录：可通过环境变量OLLAMA_MODELS指定到非系统盘（如D:\ollama-models）
- 模型迁移注意事项：必须整体复制manifests+blobs+metadata目录，任何文件缺失都会导致模型不被识别（见Bug6）
- 识别校验：ollama list 或 GET /api/tags 确认模型可见

11. 可复现性论证

- 所有依赖均为开源免费组件（FastAPI/SQLite/Ollama/Qwen/bge），无商业授权费用
- 提供完整README复现指南，新环境10分钟即可跑通
- 模型权重均为开源版本，支持国内镜像下载，无需特殊网络
- 使用轻量SQLite数据库，无需额外部署数据库服务
- 项目代码完整托管于GitHub/Gitee，支持一键克隆复现

复现步骤摘要：安装Python/Node/Ollama → 克隆项目 → ollama pull qwen2.5:3b → 后端 pip install -r requirements.txt 后 uvicorn main:app（8000）→ 前端 npm install 后 npm run dev（5173）→ 浏览器访问 localhost:5173 → 用 admin/teacher 预设账号或注册学生登录验证（完整命令见README）

12. 安全设计

- **本地部署**：全部组件（数据库、模型、服务）运行在本地，校园资料不出校门，从根源上避免云端泄露风险

- **三角色鉴权**：登录Token（Bearer）鉴权 + 后端角色依赖（get_current_user/teacher/admin）双校验，前端路由守卫防越权
- **密码安全**：随机盐+SHA256哈希存储，不存明文；修改密码需校验原密码
- **私有内容隔离**：私有问答与私有备注任何角色（含管理员）不可见，硬性规则在后端查询层强制
- **越权防护**：接口层按角色校验 + 前端路由守卫双层拦截，直接构造URL访问受保护页也无法通过
- **文件名安全**：上传文件以UUID重命名存储，原始文件名仅存数据库，防止路径穿越等注入攻击
- **依赖安全**：所有依赖锁定在requirements.txt，无高危漏洞版本

13. 未来展望

- 基于管理员人工修正的分类数据，对小模型做持续微调，提升分类准确率
- 接入GPU推理加速，提升大模型响应速度
- 增加文档段落级溯源，标注回答来自文档的具体位置
- 支持更多文档格式（Excel、Markdown、扫描件OCR）
- 增加操作审计日志与系统监控面板
- 接入云端大模型API（DeepSeek等）作为本地模型的降级方案

14. 多用户账号与三角色权限体系

14.1 角色模型

角色	获取方式	核心权限
学生（student）	自行注册	AI问答、问答历史留痕、备注、三档权限、分享链接、个人主页、修改密码
老师（teacher）	高级管理员创建	AI问答、文档管理（上传/删除/改分类/版本回滚）、删除与本人上传文档关联的公开问答、给学生公开问答写评语
高级管理员（admin）	预设固定账号，不开放注册	用户管理（新增/删除老师账号，级联清理）、公开内容总管理（查看/删除全部公开问答）

14.2 权限矩阵（硬性规则：任何角色不能查看他人私有问答、私有备注）

操作	学生	老师	高级管理员
查看自己的问答历史	✓	✓	✓
管理自己的问答（备注/权限/删除）	✓	✓	✓
查看他人私有问答	✗	✗	✗
查看/删除学生公开问答	✗	仅本人上传文档关联	✓（全部）
给学生公开问答写评语	✗	仅本人上传文档关联	✗
上传/删除/改分类文档	✗	✓	✗

新增/删除老师账号	✗	✗	✓

14.3 认证与加密

- **密码加密**：随机盐 + SHA256 哈希存储（格式 salt:digest），数据库不存明文
- **登录态**：登录返回 token（uuid），存 users 表，前端 Authorization: Bearer <token> 携带
- **后端鉴权**：get_current_user / get_current_teacher / get_current_admin 三个依赖，按接口角色校验
- **前端路由守卫**：未登录访问受保护页跳登录页；越权访问跳回本人角色首页；已登录访问登录页自动跳转
- **预设账号**：admin/admin123（高级管理员）、teacher/teacher123（老师），仅不存在时创建

15. 问答历史与三档权限分享

15.1 问答记录表（qa_records）

字段	说明
user_id	提问用户
question / answer	问题与回答原文
sources / source_doc_ids	引用来源文档（名称+ID），用于溯源与归属校验
permission	三档权限：private（私有）/ link（仅链接）/ public（公开）
note	用户备注（跟随问答权限可见性）
teacher_note	教师评语（教师对自己文档关联的公开问答所写，与学生note相互独立）
share_token	分享链接唯一标识（UUID）

15.2 三档权限机制

- **私有**：仅提问者本人可见，不出现在任何公开位置
- **仅链接可见**：任何人凭分享链接（/qa/share/token）免登录查看，不出现在个人主页
- **公开**：展示在提问者个人主页（/u/userId），任何人可访问
- **硬性规则**：私有问答与私有备注任何角色（含管理员）不可见

15.3 分享与主页流程

学生提问后自动留痕（默认私有）→ 在"问答历史"页可写备注、修改权限、生成分享链接 → 设为公开后自动出现在个人主页 → 他人通过分享链接或个人主页免登录浏览。

16. 教师文档管理与管理员后台

16.1 文档管理归属迁移

文档管理从"管理员密钥"模式迁移为**教师角色 + 登录Token鉴权**：上传接口记录文档归属（documents.teacher_id），删除/改分类/版本回滚接口均校验当前登录用户为老师角色。文档归属字段同时用于"教师删除关联公开问答"的校验。

16.2 教师删除关联公开问答

教师调用 DELETE /qa/{id} 时，后端校验：①问答必须为公开权限；②问答引用的来源文档（source_doc_ids）中至少一个属于该教师上传（documents.teacher_id == 教师ID），否则拒绝。保证教师只能清理自己文档产生的公开问答。

16.3 管理员后台能力

- 查看全部用户（学生/老师/管理员）与账号状态
- 新增老师账号（用户名+密码，校验唯一）
- 删除账号（直接删除并级联清理该用户的文档与问答；不能删除自己与admin角色）
- 查看/删除全部用户的公开问答与公开备注

16.4 教师评语机制

qa_records表新增teacher_note字段（教师评语），与学生自己的note备注相互独立：学生备注是自己的学习记录，教师评语是老师对公开问答的批改意见。教师通过 PUT /teacher/qa/{qa_id}/note 写入评语（归属校验：该问答引用的来源文档必须由该教师上传），评语随问答展示在学生端全部位置（问答历史、个人主页、分享落地页），形成"学生提问 → 教师点评"的教学闭环。

17. 前端工程化改造（Vue3）

17.1 技术架构

前端从单页Bootstrap重构为 **Vue3 + Vite + Pinia + VueRouter + Element Plus** 多页面工程（frontend/ 目录），与后端FastAPI前后端分离。

17.2 路由与页面结构

路由分组	页面	鉴权
公共	登录 /login、注册 /register、分享落地 /qa/share/:token、个人主页 /u/:userId	免登录
学生	AI问答 /student、问答历史 /student/history、个人主页 /student/profile、个人设置 /student/settings	登录+student
老师	AI问答 /teacher、公开内容管理 /teacher/manage	登录+teacher
管理员	后台首页 /admin、用户管理 /admin/users、公开内容管理 /admin/public	登录+admin

17.3 工程组织

- src/views：按角色组织的页面组件（AiChat / Login / Register / History / Profile / Settings / TeacherManage / AdminHome / UserManage / AdminPublicManage / ShareView / PublicProfile）
- src/stores/user.js：Pinia 登录态（token + role），getters 提供 isStudent / isTeacher / isAdmin
- src/router/index.js：全量路由 + 全局前置守卫（登录校验 + 角色越权拦截）
- src/api/index.js：统一接口封装，自动携带 Authorization 头
- AiChat.vue：原生fetch + TextDecoder 流式解析SSE（打字机渲染），asking状态锁防重复提交

- vite.config.js：开发模式代理后端接口到 8000，host 0.0.0.0 支持局域网访问